IDENTITY GOVERNANCE VALIDATION ENGINE

Architecture Reference

Microsoft Entra ID · Azure Functions · PowerShell

IAM Engineering Portfolio

Contents

1. Introduction

Enterprise IAM environments accumulate risk silently. Accounts persist after employees leave. Contractors end up in manager-tier groups. Azure Owner roles get assigned directly to users instead of through groups. Privileged accounts go without MFA. Entra ID records this state; it does not evaluate it.

The Identity Governance Validation Engine is a PowerShell-based compliance and risk assessment system built for Microsoft Entra ID and Azure. It reads identity and RBAC state from the Microsoft Graph and Azure ARM APIs, evaluates every identity against a configurable rule set, and answers two questions deterministically: is this identity compliant, and how much risk does it represent.

It runs in two contexts. As a standalone governance tool, it executes scheduled or on-demand tenant scans. As a control embedded in a Joiner-Mover-Leaver (JML) identity lifecycle pipeline, it evaluates a proposed identity change before it happens and verifies the actual result after. Both contexts share the same rule engine, the same finding schema, and the same risk model; only the collection step differs.

Non-goals. The engine never modifies an identity, group membership, or role assignment; it reads state only. It does not replace Entra ID Protection or Microsoft Defender for Identity, which detect attack patterns; this evaluates configuration compliance against a defined entitlement model. It does not process real-time signals; every run is a point-in-time snapshot. It does not evaluate Conditional Access policy compliance, device state, or guest/B2B accounts.

2. Design Principles

The engine is built around five architectural principles. Every design decision documented later in this reference, including the five ADRs in Section 16, traces back to one of these five.

Deterministic Evaluation. The same identity snapshot evaluated against the same governance rules always produces identical findings. Evaluation never depends on external state beyond the snapshot and policy documents.

Policy as Data. Governance policy belongs to configuration, not implementation. Rules, privileged groups, severity levels, and Separation of Duties definitions are all externally supplied JSON.

Separation of Concerns. Collection, evaluation, classification, reporting, and orchestration each own exactly one responsibility.

Read-Only Governance. The engine evaluates identity state but never modifies it. Remediation belongs to downstream approval workflows.

Provider-Agnostic Snapshots. Rule processors operate against normalized identity snapshots rather than Microsoft Graph objects, allowing additional identity providers to reuse the existing evaluation engine.

3. Design Goals

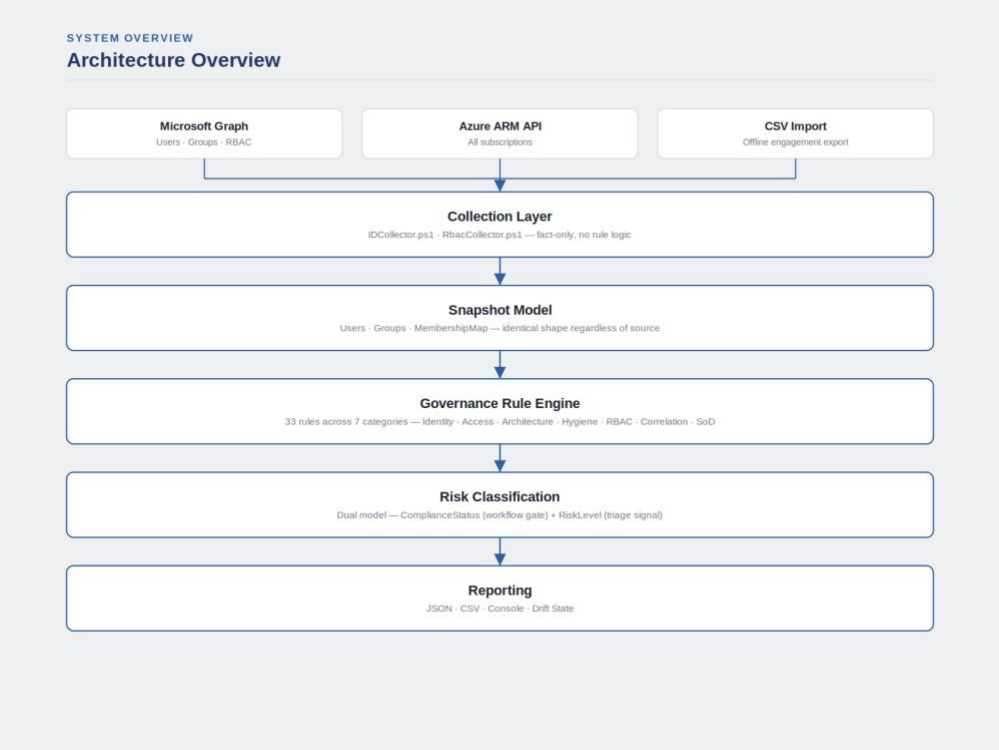
Two further commitments sit alongside the five principles above. These describe properties of the whole system rather than a single architectural choice, so they sit apart from the principles rather than duplicating them.

Auditable. Every finding traces to a named rule, a severity, and a human-readable explanation. Every report records exactly which rules were in scope for that run.

Fail informative, not silent. A missing API scope, an expired session, or a partial collection failure should be visible in the output, not absorbed into a report that looks identical to a clean result. (Section 17 covers where this goal is not yet fully met.)

4. Architecture

The engine is a five-layer pipeline. Each layer has exactly one job and cannot reach into another layer's responsibility.



Collection reads live state from Graph and ARM, or reconstructs an equivalent snapshot from CSV. It contains no rule logic; a collector's only job is to call an API, normalize the response, and return a consistent object shape.

Snapshot Model is the shared contract every collector produces and every rule processor consumes, detailed in Section 8.

Rule Evaluation applies the rules document against the collected snapshot(s) and emits findings. It makes no API calls and holds no state beyond the single evaluation pass.

Classification aggregates a flat list of findings into a per-entity risk state and a tenant-wide summary. It has no knowledge of what any individual rule means; only weights, severities, and blocking flags.

Reporting turns classification results into console output, CSV, JSON, and a drift-state file for trend comparison. It makes no decisions and modifies no data.

Orchestration, a single entry point, wires the other four layers together and is the only file permitted to call across layer boundaries. Every other file only calls into the layer directly below it.

5. Scan Modes

Validation happens at two points relative to an identity change: before it is committed, and after.

Preventive validation runs before any write to Entra ID. It answers a single question: does this proposed change violate policy. Only blocking findings matter. This is what makes the engine a gate rather than an auditor: a Joiner or Mover event that fails preventive validation never reaches the Graph API at all.

Detective validation runs against state that already exists: either a specific identity immediately after a write, or the entire tenant on a schedule. It answers a broader question: does current reality match policy, regardless of whether a JML event caused the drift. This is what catches manual changes, expired grants, and anything that happened outside the pipeline entirely.

The same rule engine, the same finding schema, and the same risk model serve both postures. What changes is only which entities are in scope and which categories of rule can meaningfully evaluate them. Four concrete modes implement these two postures:

FullScan: evaluates every identity in the tenant against the complete rule set. The standalone governance use case: a scheduled compliance sweep, or an on-demand assessment. Runs against live Graph and ARM data by default; also runs against a CSV export with no tenant connection at all (Section 7).

PreProvision: the preventive mode, evaluating a single identity before an access change happens. Runs in two forms: against a canonical identity payload for an identity that doesn't exist in Entra yet (zero Graph calls, a synthetic snapshot), or against a real object identified by its Entra ID (a fast, targeted fetch: three Graph calls regardless of tenant size). The payload form is the actual gate a Joiner or Mover event passes through before provisioning executes.

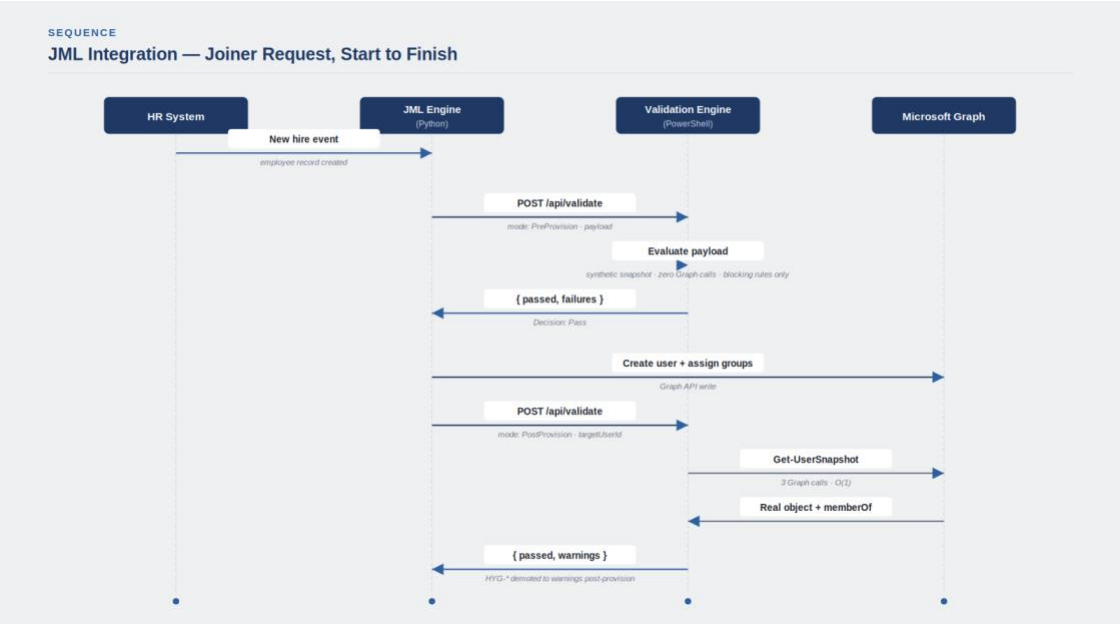
PostProvision: the detective counterpart, evaluating the same real object the payload form gated, seconds after the Graph write completes, to confirm the actual result matches what was intended. Hygiene findings that have no meaning on a freshly created account (no sign-in history yet, MFA not yet registered) are demoted to warnings rather than failures in this mode specifically. Not a distinct engine mode internally; the HTTP integration layer routes it to the same targeted-fetch path PreProvision uses.

DriftOnly: a lighter structural scan between full assessments, evaluating only Architecture, RBAC, Access, and Correlation categories. Faster to report because fewer rules run, though the underlying data collection is the same volume as FullScan.

What each mode can see differs by data plane, not by mode name. FullScan online has full Graph and ARM access. PreProvision and PostProvision have identity-plane Graph data only; a single-identity check has no meaningful "which subscriptions" question. FullScan offline (Section 7) has identity-plane data only, from a different source entirely. Every rule processor evaluates whatever data the snapshot actually contains; a rule that depends on an absent field simply never fires, rather than throwing.

6. JML Integration Flow

PreProvision and PostProvision exist specifically to serve the JML pipeline introduced in Section 1. The sequence below traces one Joiner event end to end, showing exactly where each HTTP call sits relative to the actual Graph write.



The ordering is the control. The preventive call completes entirely before any Entra object exists. The detective call runs only after the write is already committed. There is no point in this sequence where the engine's output can be bypassed without an explicit decision by the JML engine to ignore it.

7. Collectors

Four collector functions build the identical snapshot shape from four different sources. `Get-IdentitySnapshot` and `Get-UserSnapshot` read live Microsoft Graph state, at full-tenant and single-identity scope respectively. `New-IdentitySnapshotFromPayload` builds a synthetic snapshot from a canonical JML payload with zero Graph calls, for an identity that doesn't exist yet. `New-IdentitySnapshotFromCsv`, the fourth, is the offline path described in the rest of this section.

FullScan also runs as a CSV-driven assessment with no tenant connection at all; built for consulting engagements where deploying a live app registration into a customer's tenant, and leaving it there for the engagement's duration, creates a level of access a security-conscious customer is right to push back on.

The customer exports three files (users, groups, and group memberships) using their own credentials and the first-party Microsoft Graph PowerShell SDK, then hands them over. The assessment runs entirely against that export. Consent was to Microsoft, not to a third party, and nothing is left deployed in the tenant afterward.

The exported data carries user attributes, group metadata, membership relationships, and on-premises sync status; enough for the full identity-plane rule set, including hybrid identity governance. It does not carry sign-in history, password age, MFA state, PIM eligibility, or RBAC assignments. Sign-in and MFA data require live Graph scopes a point-in-time export doesn't capture continuously; RBAC assignments span arbitrary scope hierarchies across every subscription and don't reduce to a flat file the way user and group attributes do. A snapshot built from this export sets those fields to null or false rather than omitting them, so downstream rules degrade safely (no finding, not an error) on data the export was never designed to carry.

Each engagement is a self-contained folder: its own rule set, its own Separation of Duties conflict catalogue, its own input and output. The engine code itself never changes between engagements; only the data and policy it's pointed at.

8. Snapshot Model

Every collection path (live Graph, targeted single-user fetch, synthetic pre-creation payload, or CSV import) produces the identical snapshot shape: a list of users, a list of groups, a membership map, and a collection timestamp. Rule processors are written against this shape and are unaware of which path produced it, with one exception described below.

User fields include the obvious identity attributes (UPN, department, job title, employee type and ID, account enabled state) plus several load-bearing derived fields:

- **`EmploymentStatus`**: a normalized value (`Active` / `Terminated` / `Offboarded` / `Unknown` / `Guest`) computed from raw `EmployeeId` and `AccountEnabled` state, since rules should reason about lifecycle status, not reconstruct it from raw fields themselves.
- **`IsPrivileged`**: true if the user belongs to any group classified as privileged, resolved once at collection time so every rule that cares about privilege reads a single boolean rather than re-deriving it.
- **`OnPremisesSynced`**: whether the identity is authoritative in on-premises Active Directory via Entra Connect Sync. Present on both users and groups; used to detect a specific governance gap where a cloud-sensitive entitlement is anchored to an AD-controlled object (Section 11).
- **`IsPayloadScan`**: a flag on the snapshot as a whole, not the user, set only when the snapshot was built from a pre-creation payload. It gates rules that have no meaning before an identity exists: Separation of Duties evaluation, orphaned-account checks, and PIM eligibility all skip entirely when this flag is set, since none of those concepts apply to an object that hasn't been created yet.

Group fields mirror the user shape (`IsPrivileged`, `IsAssignableToRole`, `IsDynamic`, `OnPremisesSynced`), because architecture and hygiene rules increasingly need to reason about the group itself, not just who's in it.

Privilege classification uses three signals together, unioned: whether Entra marks the group role-assignable (the authoritative signal), whether the group's display name exactly matches an entry in the entitlement model whose tier is configured as privileged, and a regex pattern match as a catch-all for groups outside the model entirely (legacy `Admin`/`Tier0` naming). No single signal is trusted alone; a tier-based check alone would miss an operationally privileged group sitting at a nominally low tier.

The offline CSV path is not a full parity path. The three exported files carry user attributes, group metadata, membership relationships, and on-premises sync status; enough for the identity-plane rule categories, including hybrid identity governance. They do not carry sign-in history, password age, MFA state, PIM eligibility, or RBAC assignments, since those require live Graph or ARM access a customer-run export can't practically reduce to a flat file. A snapshot built from CSV sets these remaining fields to null or `false` rather than omitting them, so downstream rules degrade safely (no finding, not an error) rather than crashing on a missing property.

9. How Validation Works

Every run compares one identity snapshot against one rules document and produces findings. Nothing else influences the result, per the determinism principle in Section 2.

A finding is evidence that one observed identity state differs from one expected identity state. What that comparison actually looks like end to end, for one real identity, is worked through next.

10. Worked Example

The abstractions above (snapshots, rules, findings, dual-model classification) describe the mechanism. This is what they produce against one real identity in the dev tenant, unmodified from an actual FullScan run.

Four rule categories fire independently and combine into one classification. `ACCESS-003` and `ENT-003` come from the entitlement model reasoning about group membership; `SOD-EVAL-001/SOD-001` comes from the separately maintained SoD conflict catalogue; `HYG-002` is an operational signal that raises the risk score without affecting `ComplianceStatus`. The `Blocking` result is driven by the SoD finding alone; `ACCESS-003`, `ENT-003`, and `HYG-002` are all non-blocking and would not, on their own, produce that classification.

Removing either `SG_Finance_PaymentApprovers` or `SG_Finance_PaymentProcessors` from this identity's membership resolves the SoD conflict and, per the dual-model classifier in Section 12, immediately drops `ComplianceStatus` back to `Compliant`, even though the other three findings remain exactly as they are.

11. Rules

Rules are evaluated by category, each with its own processor:

Identity findings: is this account in a coherent lifecycle state? Malformed UPN, unresolved employment status, a disabled account still sitting in a privileged group, a new joiner whose HR record can't be found.

Access findings: does group membership match what the role actually requires? An intern in a manager-tier group, a terminated account that still holds access, an account holding both privileged and standard membership at once, a group nobody defined in the entitlement model.

Architecture findings: does the tenant's structural hygiene hold up? A Tier-0 asset managed by a non-Tier-0 account, an admin account also used for daily work, a service account sitting in interactive user groups, a privileged group whose membership is actually controlled by on-premises AD rather than Entra governance.

Hygiene findings: operational health signals that matter but aren't compliance failures on their own: stale sign-in, aging passwords, orphaned accounts and groups with no members. The one exception is a privileged account with no MFA registered; that's treated as a hard security control, not an operational nicety, and gates compliance the same way a real access violation would.

RBAC findings: direct Azure role assignments instead of group-based ones, privileged roles granted at too broad a scope, forbidden roles (Owner) assigned to anyone directly.

Correlation findings: risks visible only when identity and RBAC data are read together. A privileged-tier group member who also holds Owner at subscription scope. A disabled account that still has live Azure permissions. A contractor holding a role that should only ever belong to a full-time employee.

Separation of Duties (SoD) findings: group combinations that should never coexist on one identity, evaluated against a customer- or engagement-specific conflict catalogue rather than a fixed list. Payment approval and payment processing. IT provisioning and IT access approval. Finance auditing alongside the transactions being audited. A block-rated conflict stops the identity from being considered compliant; a warn-rated conflict is recorded but doesn't gate.

Rule definitions are data, evaluated at runtime by a small set of reusable evaluator functions dispatched by a `type` field; adding a new rule is a JSON entry plus, if no existing evaluator fits, one new function, never a change to the orchestration or classification layers.

Suppression. When two rules would both fire for the same underlying fact, the more specific one wins and the more general one is removed from the output; a terminated account flagged for both "in a privileged group" and "in any group" only needs to be reported once, at the more precise level. Suppression relationships are declared in the rules document, not hardcoded per rule.

Hybrid identity governance is the newest addition to this category set: two rules that detect when a cloud-sensitive entitlement (a privileged group membership, or control of a privileged group's membership itself) is anchored to an object authoritative in on-premises Active Directory rather than Entra ID. This matters because an on-premises AD compromise then becomes a direct path to privileged cloud access, with no Entra-side approval trail for however that access was granted. Both rules are purely additive: a synced identity or group is evaluated against every other rule exactly as a cloud-only one is; these two add findings, they never suppress or soften anything else.

12. Risk Classification

The engine produces two independent outputs per entity, deliberately kept separate because they answer different questions.

ComplianceStatus (`Compliant` / `NonCompliant` / `Blocking`) drives workflow decisions: can this identity proceed, does it need review, does it need immediate escalation. It's gated on compliance-relevant findings only; non-blocking hygiene findings (stale sign-in, aging passwords) contribute to the risk score but cannot, on their own, make an otherwise-clean identity non-compliant. An account can be operationally neglected and still be policy-compliant; those are different failure modes and the model doesn't conflate them.

RiskLevel (`None` through `Critical`) is an informational triage signal for security review, using the full finding set including hygiene. It never gates a workflow decision on its own.

Every entity also gets a normalized 0-100 `RiskScore`, computed as the sum of each finding's weight divided by a fixed theoretical maximum, capped at 100. The maximum is a constant, not derived from the current run's worst entity; a tenant-relative maximum would compress scores in an otherwise-clean tenant the moment one severely non-compliant identity appeared, making scores incomparable across runs and across tenants. `RiskDensity` (score divided by violation count) separates a handful of severe findings from a broad spread of minor ones carrying the same total weight.

`RecommendedAction` translates the two-model classification into a single workflow signal (none, hygiene review, alert, review, or escalate), so downstream automation has one field to branch on rather than reasoning about both models independently.

13. Reporting

A run produces console output for immediate review, and optionally CSV and JSON exports for downstream consumption. Every exported finding is enriched with human-readable context (display name, department, group memberships, tier), resolved once at the point where the identity snapshot, RBAC snapshot, and classification results all coexist, rather than making every finding carry that context redundantly.

FullScan runs can persist a drift-state file, letting the next run report whether the tenant's compliance posture is improving, worsening, or stable relative to the last baseline. The report header always states which mode ran, whether the data came from a live tenant or a CSV import, and exactly which rules were in scope; so a reader never has to guess what was and wasn't evaluated.

14. Configuration

The engine hardcodes no policy. Every governance decision (which groups are privileged, which employment types belong in which tier, which group combinations conflict, which findings block and which warn) lives in two JSON documents supplied per deployment or per engagement: a rules document defining the entitlement model and every evaluated rule, and a Separation of Duties policy document defining conflict pairs.

This split exists because both are genuinely tenant-specific. One organization's finance approver group has a different name and a different GUID than another's, and there is no portable default SoD catalogue that would mean anything across two customers. For a standalone deployment, both files live in the engine root. For a consulting engagement, each customer gets an isolated folder containing their own rules document, their own SoD policy, their input data, and their output reports; the engine code itself never changes between engagements, only the data it's pointed at.

15. Performance and Complexity

Not exact, but representative of where cost scales with tenant size.

Stage	Complexity	Notes
Identity collection	O(U + G + M)	One paginated call for all users, one for all groups, one membership call per group. Membership collection (M) dominates for tenants with many groups.
Rule evaluation	O(U x R)	Every identity-plane rule iterates every user. Correlation rules add a lookup against RBAC assignments per user, not a further multiplication.
RBAC collection	O(S x A)	One enumeration call per subscription; assignment volume per subscription varies independently of tenant size.
Classification	O(F)	Linear in finding count; one aggregation pass per entity.
Memory	O(U + G + M + F)	The full snapshot and finding list are held in memory for the duration of a run; nothing is streamed or paged to disk.

Where U = users, G = groups, M = memberships, R = rules, S = subscriptions, A = role assignments, F = findings.

Largest Bottlenecks

1. **Group membership enumeration.** One Graph call per group. Dominates total collection time as group count grows, independent of user count.
2. **Azure RBAC collection.** One ARM call per subscription. Scales linearly with subscription count, not with tenant size.
3. **Subscription traversal.** Enumerating and iterating every accessible subscription before any RBAC call can start.

These are the two costs that don't shrink with a smaller rule set; they're driven by tenant shape, not by what's being checked. This is why DriftOnly is faster to report but not faster to collect: it evaluates fewer rules against the identical collection cost.

16. Architecture Decision Records

ADR-001: Normalized Snapshot Contract Across All Collectors

Status: Accepted

Context

The rule engine needs to operate against multiple collection paths: Microsoft Graph, CSV imports, and synthetic JML payloads.

Decision

All collectors must produce the same snapshot contract.

Consequences

The rule engine becomes source-agnostic. Offline collection requires no special-case logic. Future collectors (e.g. Okta) can plug into the same interface.

ADR-002: Policy as JSON, Not Code

Status: Accepted

Context

Governance rules (which groups are privileged, which employment types are permitted where, which findings block versus warn) change for policy reasons, not engineering reasons, and change more often than the engine's own logic does.

Decision

The entitlement model, the rule set, and the SoD conflict catalogue are all externally supplied JSON documents, read at runtime. No rule condition, weight, or severity is hardcoded in PowerShell.

Consequences

A policy change is a JSON edit, not a deployment. The same engine binary runs identical logic against completely different policy per deployment or per engagement. The trade-off: a malformed or incomplete rules document is a runtime risk, not a compile-time one; the engine has no schema validation for the rules document beyond what each evaluator function checks defensively at read time.

ADR-003: Collectors Never Evaluate Rules

Status: Accepted

Context

Without a firm boundary, a "quick fix" during collection can quietly become a rule concern; for example, skipping a group during collection because it looks unimportant. Once that happens, a data bug and a logic bug can no longer be diagnosed independently.

Decision

A collector's only job is to call an API (or read a CSV) and return a normalized object. It contains no conditional logic that depends on the rules document, the entitlement model, or any policy concept.

Consequences

Collectors are testable with a mocked API response and nothing else. A misfire is always either "wrong data" or "wrong rule," never both, which halves the search space during debugging. Some computed fields (IsPrivileged, EmploymentStatus) are still resolved at the collector layer because they are derived facts rather than policy decisions; maintaining that distinction is a discipline the codebase enforces by convention, not one the language enforces structurally.

ADR-004: Rule Processors Never Call Graph

Status: Accepted

Context

A rule processor that could reach back into Microsoft Graph mid-evaluation would make a rule's outcome depend on tenant connectivity and live API state at the moment it runs, not just on the snapshot it was given.

Decision

IDRuleProcessor.ps1, RbacRuleProcessor.ps1, and CorrelationRuleProcessor.ps1 accept only a snapshot and a rules document as input. None of them import a Graph or Az module, and none make an outbound call.

Consequences

A rule can be unit-tested against a hand-built snapshot with zero network dependency. Determinism (Section 2, Principle 1) is enforced structurally rather than by convention; there is no code path through which a rule's result could depend on anything other than the snapshot and the rules document it was given.

ADR-005: Single Finding Schema Across All Rule Categories

Status: Accepted

Context

Seven rule categories (Identity, Access, Architecture, Hygiene, RBAC, Correlation, SoD) each detect a structurally different kind of problem. Left unconstrained, each category's processor could reasonably invent its own output shape suited to what it checks.

Decision

Every rule, in every category, emits the same finding shape through one constructor, New-ComplianceFinding: entity, entity type, rule ID, category, severity, weight, blocking flag, and a human-readable explanation. No processor builds a finding manually.

Consequences

Classification and reporting are written once against one shape and never change when a new rule category is added. The trade-off is expressiveness: a finding that would benefit from category-specific structured data (RBAC's scope and role, for instance) has nowhere to go except the free-text explanation, which is the coupling behind the suppression fragility already recorded in Known Limitations.

17. Known Limitations

Read-only by design. The engine detects and reports; it does not remediate. This is deliberate, not a gap; remediation is a distinct, higher-stakes responsibility with its own approval and audit requirements, and belongs in a separate layer that consumes this engine's output.

Offline assessments cannot see RBAC, sign-in activity, MFA state, or PIM eligibility. These require live Azure ARM or Graph API access that a customer-run CSV export cannot practically carry; RBAC assignments in particular span arbitrary scope hierarchies across every subscription and don't reduce to a flat, customer-exportable file the way user and group attributes do. Rules depending on these fields are out of scope for an offline engagement rather than producing false results.

A failed RBAC collection and a genuinely clean tenant currently look identical in a report. If Azure authentication has expired or subscription enumeration fails outright, the engine logs the failure to an optional error log and continues with an empty RBAC result set; the same output shape a tenant with zero RBAC risk would produce. Nothing in the report itself distinguishes "nothing to report" from "nothing was actually checked." A collection-success indicator on the report header would close this gap.

The risk-scoring cap needs recalibrating against the current rule set. The normalization constant was set against an earlier, smaller rule set and has not been revisited as heavier rules (notably the SoD engine) were added. An entity that now genuinely stacks past the old ceiling will show an identical maximum score to one that exactly reached it; the underlying raw weight sum still differentiates them, but the normalized score used for sorting and dashboards does not.

The scope-aware suppression between the two RBAC severity levels is implemented against the human-readable finding text, not structured data. It works today, but it depends on the wording of a finding's explanation never changing in a way that breaks the pattern it's matched against; a fragile coupling between a display string and a piece of logic that should be independent of it.

Employment-type normalization is implemented twice, slightly differently, in two different rule processors. Both reduce the same raw HR field to a canonical value, but they don't currently agree on what that canonical value is. No rule has yet needed both processors to agree, so this hasn't produced a visible defect; but it's exactly the kind of divergence that will eventually cause one.

18. Future Architecture

Extend validation to Okta tenants. The collector layer's only real contract is producing the shared snapshot shape: a list of users, a list of groups, a membership map. Nothing above that layer knows or cares whether the data came from Microsoft Graph, a CSV export, or a synthetic payload. That same contract extends naturally to a second identity provider: an Okta collector implementing the identical snapshot shape would let every existing rule, every classification model, and every report format work unchanged against an Okta tenant. This is the clearest path to a genuinely multi-IdP governance engine rather than an Entra-only one, and it validates the collector/processor separation as a real architectural boundary rather than a convenience.

Complete the diagram set. Collector Architecture, Snapshot Lifecycle, and Rule Engine processor-breakdown diagrams, in the same visual style as the System Overview and Worked Example diagrams above, plus a Repository Structure diagram for engineers navigating the codebase directly.

Recalculate and version the risk-scoring cap, with the normalization constant treated as a versioned parameter so a cap change doesn't silently invalidate a stored drift baseline from before the change.

Replace text-pattern suppression with structured finding metadata, so scope-aware suppression logic depends on actual data fields rather than parsing a display string.

Consolidate employment-type normalization into a single shared function, called by every processor that needs it, rather than two independently maintained implementations.

Add a collection-success indicator to the RBAC and identity snapshots, so a report can state plainly whether a given data plane was actually collected, rather than leaving that distinction to be inferred from an external error log.

SIEM integration. Findings already carry stable rule identifiers and ISO 8601 timestamps; a direct connector to a security information and event management platform is a natural next consumer of the existing output, not a redesign.

Parallelized collection for large tenants. Group membership and per-subscription RBAC calls are each independent and a natural candidate for concurrent execution once tenant size makes sequential collection a meaningful bottleneck; this has not yet been validated against a tenant large enough to need it.